Input | Output

NEAR / Midnight: Findings & Hypotheses

17 April 2026

Scope and goals

Goal	Description
TPS gains	Improve Midnight throughput, perhaps using NEAR
TEE privacy	TEE as a fast path to adding privacy, inspired by NEAR
Key management	NEAR's key system as a model for new Midnight capabilities
Chain abstraction	Derived keys for Midnight, Cardano, Ethereum, Solana, etc.
Midnight OS platform	NEAR as the substrate for Starstream, Nightstream, Paima, etc.
Feature identification	Identify NEAR capabilities that could benefit Midnight

The aim is assessment, not an architecture, roadmap, or specification.

Option	Approach	Cost
1 — Port	Full re-platforming onto the NEAR stack	Very high
2 — Incorporate software	Extract specific NEAR components	Medium-high
3 — Incorporate ideas	Rebuild NEAR-inspired patterns natively	Medium

Scope and goals

Goal	Description
TPS gains	Improve Midnight throughput, perhaps using NEAR
TEE privacy	TEE as a fast path to adding privacy, inspired by NEAR
Key management	NEAR's key system as a model for new Midnight capabilities
Chain abstraction	Derived keys for Midnight, Cardano, Ethereum, Solana, etc.
Midnight OS platform	NEAR as the substrate for Starstream, Nightstream, Paima, etc.
Feature identification	Identify NEAR capabilities that could benefit Midnight

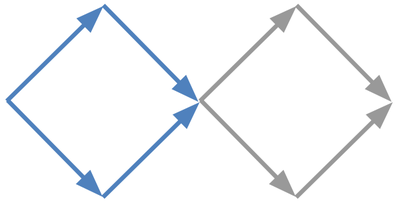
The Midnight Passport document covers several aspects in detail.

ARC already has work underway in these areas.

The aim is assessment, not an architecture, roadmap, or specification.

Option	Approach	Cost
1 — Port	Full re-platforming onto the NEAR stack	Very high
2 — Incorporate software	Extract specific NEAR components	Medium-high
3 — Incorporate ideas	Rebuild NEAR-inspired patterns natively	Medium

Process



Increment	Diamond	Phase	Days	Dates	Focus
1	Foundation	Divergent	1–25	Mar 12 – Apr 5	*Broad exploration:* generate hypotheses
2		Convergent	26–50	Apr 6 – Apr 30	*Hypothesis testing:* converge on recommendations
3	Refinement	Divergent	51–75	May 1 – May 25	*Refined exploration:* regenerate hypotheses
4		Convergent	76–100	May 26 – Jun 19	*Hypothesis testing:* converge on recommendations

LLM-friendly repository https://github.com/input-output-hk/arc-nearfall-evaluation 🔒

- Project plan
- Experiments
- Assessments
- Findings
- Artifacts
- Lessons learned
- Designs
- Daily journal
- Weekly reports
- Apps
- Background lit
- NotebookLM

Index of assessments: *NEAR & Midnight*

Assessment	Topic	Key Finding
midnight-pallets	Architecture	Midnight combines standard Substrate pallets with custom ones (Kachina, Dust, Minotaur) for ZK verification, fee batteries, and Cardano anchoring
midnight-ledger-notes	Architecture	The four-part ledger is very cleanly separated from the other substrate pallets.
modularity-comparison	Architecture	Substrate is more modular than NEAR on virtually every axis; Option 1 (full port) carries highest risk due to unvalidated 300 TGas ZK-verification limit
contract-to-contract-calls	Architecture	Midnight will eventually support synchronous intra-transaction contract calls; NEAR uses asynchronous inter-block receipts — a fundamental architectural difference
midnight-vs-near-tps	Throughput	Midnight's binding constraint is block size, not proof verification; 500+ TPS may be achievable within Substrate via block-size increases and proof folding
midnight-to-near-mapping	State mapping	Per-UTXO/note accounts rank highest for throughput; coarse-grained options shift the 500+ TPS problem entirely to a bespoke off-chain sequencer
near-key-management	Cryptography	NEAR uses human-readable hierarchical accounts with multiple fine-grained access keys per account, stored directly in the state trie
near-scheduling	Validator ops	NEAR's multi-role validator model distributes load finely but requires significantly higher hardware (48 GB RAM, 2–3 TB storage) than Midnight validators

Index of assessments: *Throughput*

Assessment	Topic	Key Finding
substrate-throughput-techniques	Throughput	No existing Substrate chain demonstrates >50 TPS for ZK-proof-bearing transactions
throughput-constraint-comparison	Throughput	NEAR and Midnight have inverse architectures: NEAR is overhead-heavy with per-shard parallelism; Midnight is payload-efficient with less horizontal scaling
throughput-hypotheses	Throughput	Block size increase to 5 MB alone reaches ~49 TPS (compute becomes the new bottleneck); the 500 TPS target requires aggressive experimentation
utxo-sharding	Throughput	L1 UTXO sharding fits poorly for Midnight; proof folding (ProtoGalaxy) and intra-node parallel proof verification are possible scaling levers
utxo-pallets	Throughput	No production-grade general-purpose UTXO pallet exists for Substrate/FRAME; Tuxedo is the most serious effort but is not production-ready and loses the entire FRAME ecosystem
batching-vs-folding	Throughput	AlephBFT replaces only GRANDPA; AURA block production is unchanged, giving a significant finality latency improvement with no raw TPS gain
consensus-comparison-alephbft-aura-grandpa	Throughput	the primary gain is potential subsecond finality with no raw TPS increase; it is orthogonal to, and composable with, ProtoGalaxy folding

Index of assessments: *Privacy*

Assessment	Topic	Key Finding
chain-abstraction-vs-bridge	Wallets	NEAR Chain Signatures authorises native-asset transactions on foreign chains without deploying infrastructure there; Midnight could offer a strictly stronger variant in which the signing request is hidden in ZK private state (invisible to on-chain observers)
tee-cheat-codes	Privacy/proving	Options available for Midnight, but crypto incompatibilites need resolving
tee-vs-zk	Privacy/proving	TEE can serve two distinct, complementary roles, with ZK still providing strong privacy primitives: ● a remote proof server (no protocol change, ZK proof still generated inside the enclave) ● a first-class alternative proof mode (may require protocol change, enables mobile participation with small attestations instead of expensive ZK proofs)
starstream-nightstream	Execution/proving	Starstream provides UTXO-native WASM execution via coroutines; Nightstream is post-quantum lattice IVC; together they could form a “MidnightOS”, but the Nightstream outer SNARK appears to be still WIP
paima-on-midnight	Integration	Paima could run on Midnight’s public layer with moderate effort; privacy integration via ZK attestations is hard due to tension between deterministic re-execution and private state

Summary of experiments

Experiment	Platform	Key Finding
test-contract-1	Midnight preprod	Basic wallet operations (mnemonic → sync → transfer) work end-to-end; documentation was incomplete and partly obsolete
midnight-env pubnet-node	Midnight preprod and mainnet	Node deployment required piecing together multiple outdated sources; official docs are incorrect in several aspects; block fetching was not reliable; patch node and repackaged as a kube
node-lan	Midnight (local 7-node cluster)	Consensus was fragile; BEEFY keys apparently cannot be pre-loaded; difficult to work with documentation
shielding-contracts	Midnight preprod	Shielded mint and transfer work, but the SDK has multiple non-obvious shortcomings requiring workarounds (notably a WASM panic in ZswapChainState.tryApply); experienced Lace bug
compact-named-accounts	Midnight preprod	Stealth meta-address registry works end-to-end; however, publishing the ephemeral key on-chain leaks existence of payment events — neither path provides full graph privacy
mn-tui ★	Midnight	Fully functional textual user interface for managing Midnight wallets, contracts, minting, dust, etc.; discovered Lace bug.
local-tee-poc	Midnight + mobile	Notional KYC dapp with proof server running on a mobile device; SEV-SNP TEE demonstration; discovered packaging bug in proof server's docker.
near-exploration	NEAR testnet	NEAR developer tooling is significantly smoother than Midnight's; account creation, contract deployment, and RPC node setup all work with minimal friction
starstream-compile	Starstream (local)	Starstream compiler produces valid WASM from .star source; the language is functional but early-stage
paima-game-templates	Paima engine	Paima is a real, usable multi-chain rollup engine; templates are game/app oriented, not ZK-proof-based settlement; template was outdated

Midnight compatibility with NEAR

Although Midnight has a clean and separable interface to other Substrate pallets, migrating it to NEAR likely does not provide compelling benefits, and there are significant downsides.

However, a hybrid-sharding approach to Midnight may be viable for achieving moderate TPS.

Design philosophies

Attribute	Substrate / Midnight	NEAR Protocol
Stated goal	Modular blockchain framework	High-performance sharded L1
Primary modularity axis	Vertical component decomposition (pallets)	Horizontal state decomposition (sharding)
Codebase structure	Framework (`substrate`) + application (`midnight-node`)	Monolithic (`nearcore`) + contract layer
Customization model	*First-class:* swap pallets, runtimes, consensus	*Second-class:* extend via smart contracts mostly
Core design tension	Modularity vs. performance	Performance vs. customizability
Bookkeeping	Primarily UTxOs, but accounts / commitments for contracts	Accounts

Consensus pluggability

	Substrate / Midnight	NEAR
Mechanism	Independent pallets (AURA, BABE, GRANDPA, BEEFY)	Integrated Nightshade/Doomslug — not swappable
Swap cost	*Low:* reconfigure FRAME, recompile	*Very high*: fork `nearcore`
Midnight example	Replacing GRANDPA with AlephBFT	N/A
Assessment	✅ Genuinely modular	❌ Monolithic

Runtime / execution pluggability

	Substrate / Midnight	NEAR
Mechanism	WASM runtime, forkless upgrades via on-chain governance	WASM contract execution within fixed runtime
Upgrade path	Runtime logic upgradeable without hard fork	Protocol upgrades require node software release
Custom VMs	Possible: new pallet can introduce new host functions	Not supported at protocol level
Midnight example	`Impact` interpreter lives in WASM runtime, upgradeable	An `Impact`-equivalent would be a gas-constrained WASM contract
Assessment	✅ Strong forkless upgrade path	⚠️ Contract-layer extensible; protocol-layer rigid

State storage pluggability

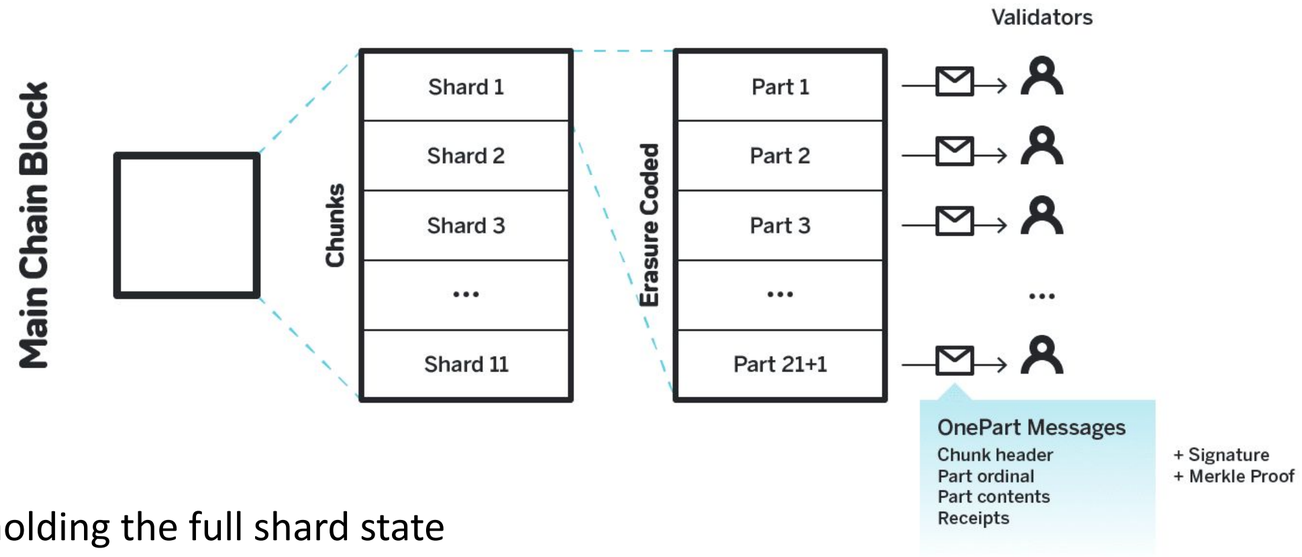
	Substrate / Midnight	NEAR
Storage model	SCALE-encoded Merkle-Patricia Trie; Hoarfrost wraps it	Merkle Patricia Trie, partitioned by shard
State isolation	Single global trie (no native sharding)	Per-shard tries composing a global trie
Custom state structures	Possible via pallet storage items and host API	Fixed per-account key-value store
ZK-specific state	Hoarfrost adds commitment trees, nullifier sets, O(1) cloning	Not natively supported; would require contract-level state
Assessment	✅ Extensible via host API versioning	⚠️ Rigid per-account model; ZK structures non-trivial to add

Scaling model

	Substrate / Midnight	NEAR
Native approach	Single-shard L1; scaling via co-chains and L2 (e.g., Midnight City)	Native sharding: Nightshade splits state/execution across shards
Shard count	N/A at L1; co-chains are separate Substrate nodes	Currently 9+ shards, planned to scale dynamically
Cross-shard latency	N/A at L1; co-chain settlement adds multi-block latency	~1 second per shard hop via receipt routing
State partitioning	Manual (each co-chain has its own state)	Automatic (accounts assigned to shards by ID)
Assessment	⚠️ Scaling requires explicit co-chain architecture	✅ Native; sharding is a first-class protocol feature

NEAR's high-throughput architecture

- *Core idea:* horizontal scaling — split global state and execution across parallel shards
- Nightshade Sharding
  - Global state divided into N shards
  - Each shard owns a subset of accounts and contracts
  - Each block contains one “chunk” per shard, produced by that shard's assigned validators
  - Validators track only their assigned shard, not the full global state
- Protocol supports dynamic resharding
- Receipt-Based Async Execution
  - Receipts can spawn further receipts
  - No synchronous cross-shard calls
  - Transactions are not atomic, but receipts are
- Stateless Validation (Nightshade 2.0)
  - Chunk producer executes state and emits a state witness
  - Merkle proofs for every trie read/write
  - Chunk validator committee verifies the witness without holding the full shard state
- Decouples storage requirements from validation



Concern about NEAR archival storage

Storage category	Mainnet archival	Testnet archival	Notes
Total	~118 TB	~16 TB	Split-storage architecture (nearcore 1.35.0+)
Cold storage	~115 TB	~15 TB	Spinning disk / cheap persistent volumes
Hot storage (SDD)	~3 TB	~1.5 TB	Recent epochs; fast random access
Transaction/receipt data	**~5–10 TB**	—	~1–2 KB/tx × 5.25 B transactions
State snapshot overhead	**~108–113 TB**	—	***Dominant cost — not transaction history***

- NEAR requires expensive archival nodes that currently lack a practical cost-recovery model.
- Snapshots of ledger state dominate storage (and perhaps network) resources.
- Validators prune blocks more than 5 epochs (60 hours) old

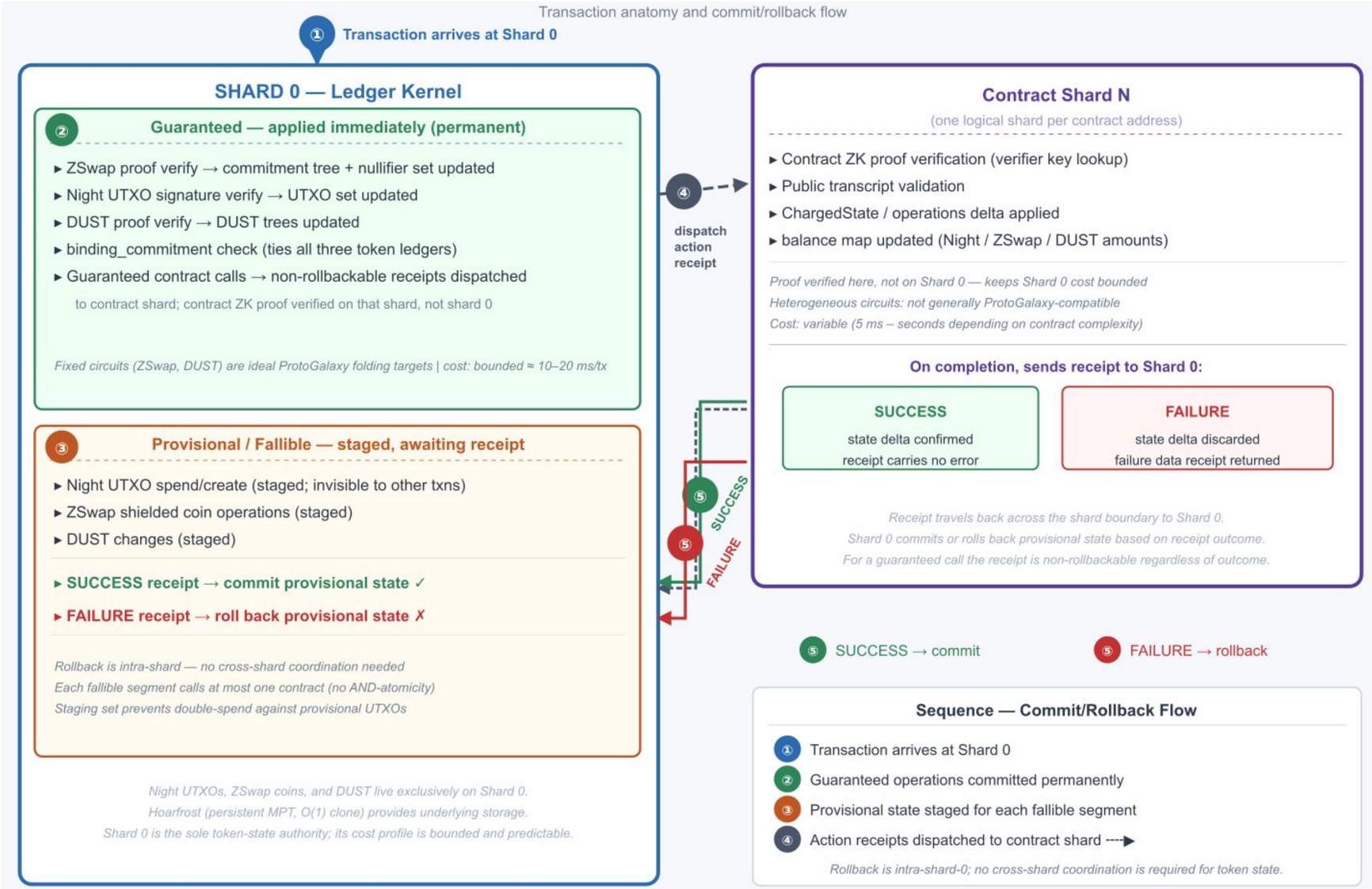
Speculative design for Midnight-on-NEAR sharding

Ledger operations occur on a special shard 0.

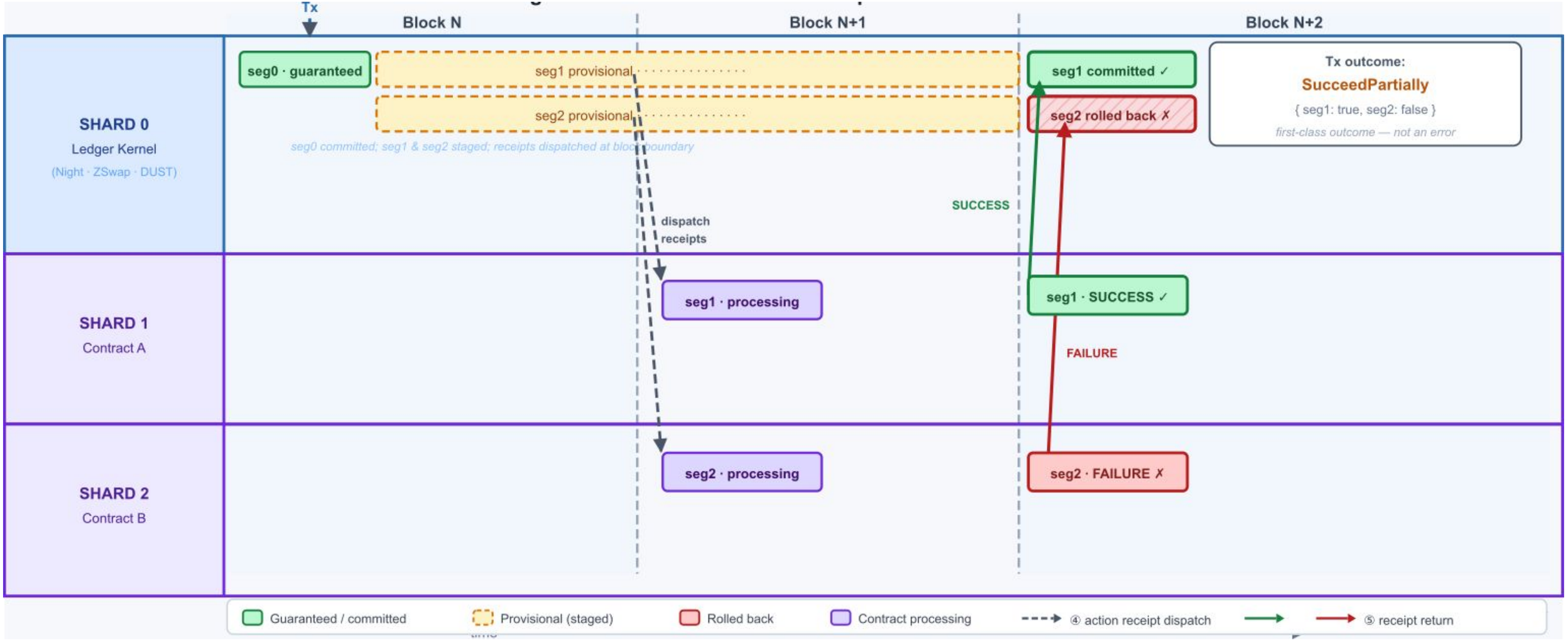
- Handles UTxOs.
- Handles token movements.
- Circuits are fixed, uniform, and cheaper: ProtoGalaxy?
- Potential bottleneck above ~100 TPS

Contract operations occur on shards > 0.

- Supports complex circuits
- Supports large public states
- Horizontally scalable



Midnight-on-NEAR: Cross-shard receipts and partial rollback



Findings / hypotheses

1. It may be possible to partially shard Midnight onto NEAR's blockchain layer (or just its algorithms) via a hybrid-sharding technique.
2. Otherwise, NEAR's architecture and codebase do not generally provide sufficient enhancements to justify migrating Midnight to it
3. Subjectively, NEAR's architecture and design seem less demonstrably robust compared to Substrate.
4. NEAR's transactions are *not atomic*: they may partially fail and/or partially roll back.
5. NEAR has an excessive storage burden for frequent snapshots of ledger state.

TPS Gains

NEAR does not overcome Midnight's data-rate bottleneck, but adjusting protocol parameters on a more reliable foundation (TCP/QUIC tuning and AlephBFT) and using ZK batching and/or folding promises improvement beyond 100 TPS and warrants experimentation and benchmarking.

NEAR's scalability achievement

Property	Value
Block time	1 s
Intra-shard finality	~1-2 s
Cross-shard finality	2+ blocks (~2 s minimum per hop)
Throughput scaling	Linear with shard count (theoretical)
Practical mainnet TPS	Demand-limited at current load (well below 4000 TPS capacity) and 9 shards

Tradeoffs:

- *Fundamental architectural constraint: async receipts mean cross-shard composability carries inherent latency.*
- *Transactions are not atomic.*
- *Heavy reliance on snapshotting and archiving.*

High-throughput substrate chains

Chain	Consensus	TPS (measured/claimed)	Block time	Notes
Aleph Zero	AlephBFT (DAG+BFT)	~89,600 (2021 prototype, 112 nodes)	~0.4 s finality	Golang prototype; ZK-shielded (LIMINAL) ~10–50 TPS
Polkadot + parachains	BABE+GRANDPA+ Parachain	143,343 aggregate (Spammening 2024, ~23% of cores)	~6 s relay	On Kusama canary network; single parachain ~1,000–2,000 TPS
Acala	AURA+GRANDPA (parachain)	~200 TPS	~12 s	DeFi ops; relay-chain bound pre-async-backing
Moonbeam	AURA+GRANDPA (parachain)	~30–60 TPS max; ~0.7 TPS mainnet avg	~6 s	EVM compatibility traded for throughput
Astar	AURA+GRANDPA (parachain)	~1,000 TPS claimed WASM; mainnet ~100–500	~6 s	EVM + WASM; ZK Dapp support planned

- While several ZK blockchains have the technical capacity to exceed 100 TPS, sustained organic demand for ZK-proof-bearing transactions in production rarely exceeds a network-wide average of 50 TPS over long time horizons.
- Many of the publicized high TPS ZK chains involve L2 sequencers or batch aggregation.
- It is only since the rollout of next-generation proof systems and data availability upgrades in 2024 and 2025 that high-throughput ZK execution has become both technically and economically viable for public mainnets.

Lessons from Ouroboros Leios

Area	Finding
Full Leios sharding	Community found required features undesirable: failed transactions recorded in blocks; fees/collateral forfeit on conflict; DoS risks from sharding; shard-selection tokens or flavored UTxOs; cascading impacts to indexers, wallets, and backends
TCP multiplexing	Praos's multiplexing of mini-protocols over TCP created problems with timely delivery of Praos blocks in the Leios prototype
Simulator optimism	Two Leios simulators (low-fidelity and medium-fidelity for TCP) may be overly optimistic about throughput achievable in a more realistically engineered prototype
Testing infrastructure	Either actual large-scale deployments or sophisticated network-emulation infrastructure (e.g. Antithesis) is needed to study real node behavior at high throughput; simple network-manipulation tools are inadequate
Tail risk	Latency/bandwidth tail risks are significant for maintaining throughput and/or consensus
Design space	Both network and CPU resources are involved in the design space; such tradeoffs can be studied with constraint modeling

Midnight's binding constraint

Resource	Limit per block	Ceiling at 6 s slot
Block size	200 KB	~3.3 TPS (200 KB ÷ ~10 KB/tx) ← **binding**
Compute budget	1 s (single-threaded)	~49 TPS (1,000 ms ÷ 3.4 ms/proof ÷ 2 proofs/tx) but number of proofs per tx varies
Proof verification	~3.4 ms constant + ~3.4 µs/public-input	

- Block size is the binding constraint — roughly 15× tighter than the compute ceiling.
- Switching to NEAR's networking does not address the bottleneck.
- ZK blockchains typically do not sustain more than ~50 TPS for ZK-proof-bearing transactions in production.

Applicability of throughput techniques

Technique	Impact on TPS	ZK-compatible?	Cost	Verdict
Block size increase (parameter)	~3–25×	Yes	Trivial	Immediate win; already identified; enabled by TCP/QUIC improvement
Shorter AURA slot time (parameter)	~2–3×	Yes	Low	Multiplies block-size gain; evaluate jointly
ZK proof aggregation (ProtoGalaxy)	~10× on compute ceiling	Yes (ZK-native)	High	High ROI; non-trivial; blocked pending security audit
Batch proof verification host function	~N× on compute ceiling	Yes (native Rust)	Medium	Architecturally clean for Midnight; not yet in Substrate
AlephBFT consensus (instead of GRANDPA)	~3–5× (slot reduction)	Yes (consensus only)	Medium –High	Open-source; subsecond finality; no runtime changes
Parallel extrinsic execution	Potentially large	N/A	N/A — does not exist	Not available in Substrate
External ZK pallet (pallet-plonk, zkVerify)	Negative (slower)	Inferior	—	Do not adopt; native host function is faster

TCP kernel tuning

Setting	Try now?	Try at 5 MB blocks?	Scope
`tcp_slow_start_after_idle = 0`	**Yes — immediately**	Yes	**Docker** `--sysctl`
`tcp_congestion_control = bbr`	**Yes — immediately**	Yes	**Docker** `--sysctl`
`default_qdisc = fq`	**Yes — immediately**	Yes	**Docker** `--sysctl` (Linux ≥ 4.18)
`tcp_mtu_probing = 1`	Yes (hygiene)	Yes	**Docker** `--sysctl`
`tcp_max_syn_backlog = 8096`	Yes (hygiene)	Yes	**Docker** `--sysctl`
`tcp_rmem` & `tcp_wmem_max = 8 MB`	Not required yet	**Yes — prerequisite**	**Docker** `--sysctl` (Linux ≥ 4.15)
`rmem_max` & `wmem_max = 8 MB`	Not required yet	Yes — if node calls `setsockopt`	**Host root** `sysctl.d`
`vm.swappiness = 1–10`	Yes (trie I/O stability)	Yes	**Host root** `sysctl.d`
`vm.dirty_ratio = 10–20`	Yes (trie I/O stability)	Yes	**Host root** `sysctl.d`
`net.core.netdev_max_backlog` increase	Yes (hygiene)	Yes	**Host root** `sysctl.d`
`fs.file-max` increase	Yes (operational)	Yes	**Host root** `sysctl.d`
`ulimit -n` (open file descriptors)	Yes (operational)	Yes	**Docker** `--ulimit nofile=65536:65536`

QUIC transport for Midnight Nodes

Property	TCP + Yamux (current)	QUIC (via litep2p)
Head-of-line blocking	All streams stall on any packet loss	Streams independent; lost packet blocks only its stream
Handshake latency	2.5 RTT (TCP + TLS 1.3)	1 RTT new; 0-RTT resumption for known peers
Slow start after idle	`tcp_slow_start_after_idle` resets CWND; ~700 ms penalty at 5 MB	Larger initial CWND + pacing; structurally absent
Stream multiplexing	Yamux over TCP; large block transfer delays GRANDPA votes	Native independent streams; GRANDPA unaffected by concurrent block download
NAT traversal	DCUtR hole punching	Connection migration via connection IDs; survives IP change

Throughput-technique risk summary

Technique	Primary liveness risk	Primary safety risk	Centralisation pressure
Block size increase	Diffusion failure at large sizes / short slots	DoS amplification; no native erasure coding	High (favours high-bandwidth validators)
Shorter slot time	Tightened diffusion window; higher fork rate	Higher steady-state fork exposure; bridge complexity	Moderate (requires low-latency nodes)
(5 MB, 2 s) combined	Marginal $\Delta_{diff}/\Delta_{slt}$ ratio; tail validators excluded	Compounded fork + diffusion risks	High — consider (5 MB, 3 s) as safer initial target
ProtoGalaxy folding	Aggregator as new SPF; batching latency at low load; circuit fragmentation	Folding-relation soundness; independent audit required	Low (aggregator can be decentralised)
Batch proof verification	Thread contention with networking under full load	Batch randomness must be unpredictable to submitters; weight model re-calibration	Low (purely internal to node)
AlephBFT consensus	DAG round failure if committee members offline	< 1/3 adversarial threshold; newer codebase	Moderate (rotating committee)

Findings / hypotheses

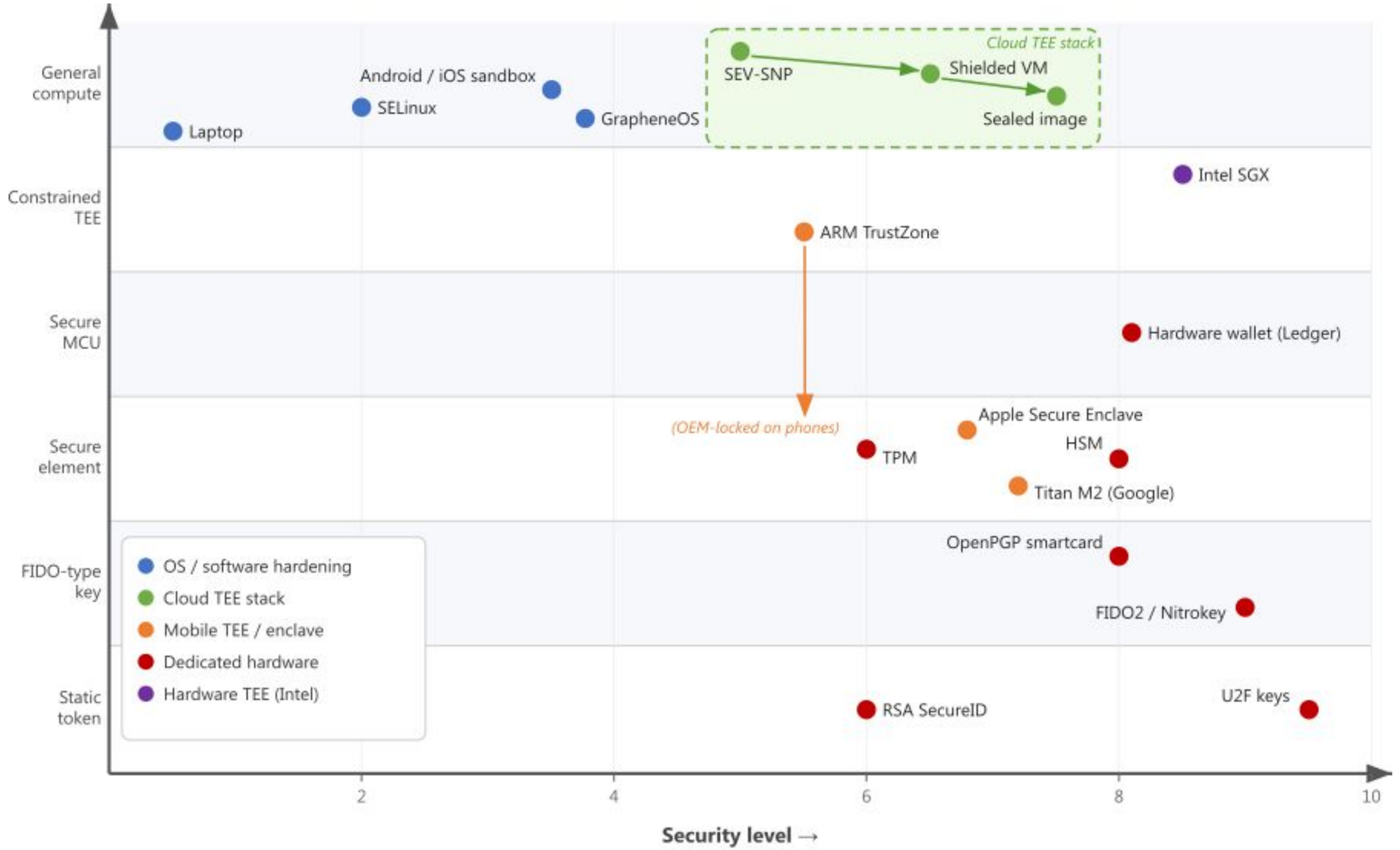
1. Empirical TPS measurements and experiments, though costly in terms of labor and computing, are essential even in early stages of a potential throughput upgrade.
2. Apparently, there are no general-purpose network simulators/emulators available for Substrate, though a tool like Shadow or Antithesis might be useful.
3. Conceivably, the following could cumulatively support 150+ TPS.
    - Block size increase (increased data rate)
    - Shorter slot time (increased data rate)
    - TCP tuning (increased reliability)
    - QUIC instead of TCP (enables larger blocks)
    - AlephBFT instead of GRANDPA (enables shorter slot times)
    - Batch proof verification (quickens verification)
    - Protogalaxy folding (removes CPU bottleneck)
4. Sharding approaches will require intensive involvement of Research (cf. Leios).

TEE* “Cheat Codes”

If a TEE authorises private asset movements by emitting an opaque signed token and the ZK circuit consumes the token as a public input, then this keeps ZK witness complexity bounded and independent of application logic. Such small bounded circuits can execute quickly on mobile devices, for example.

* TEE = Trusted Execution Environment

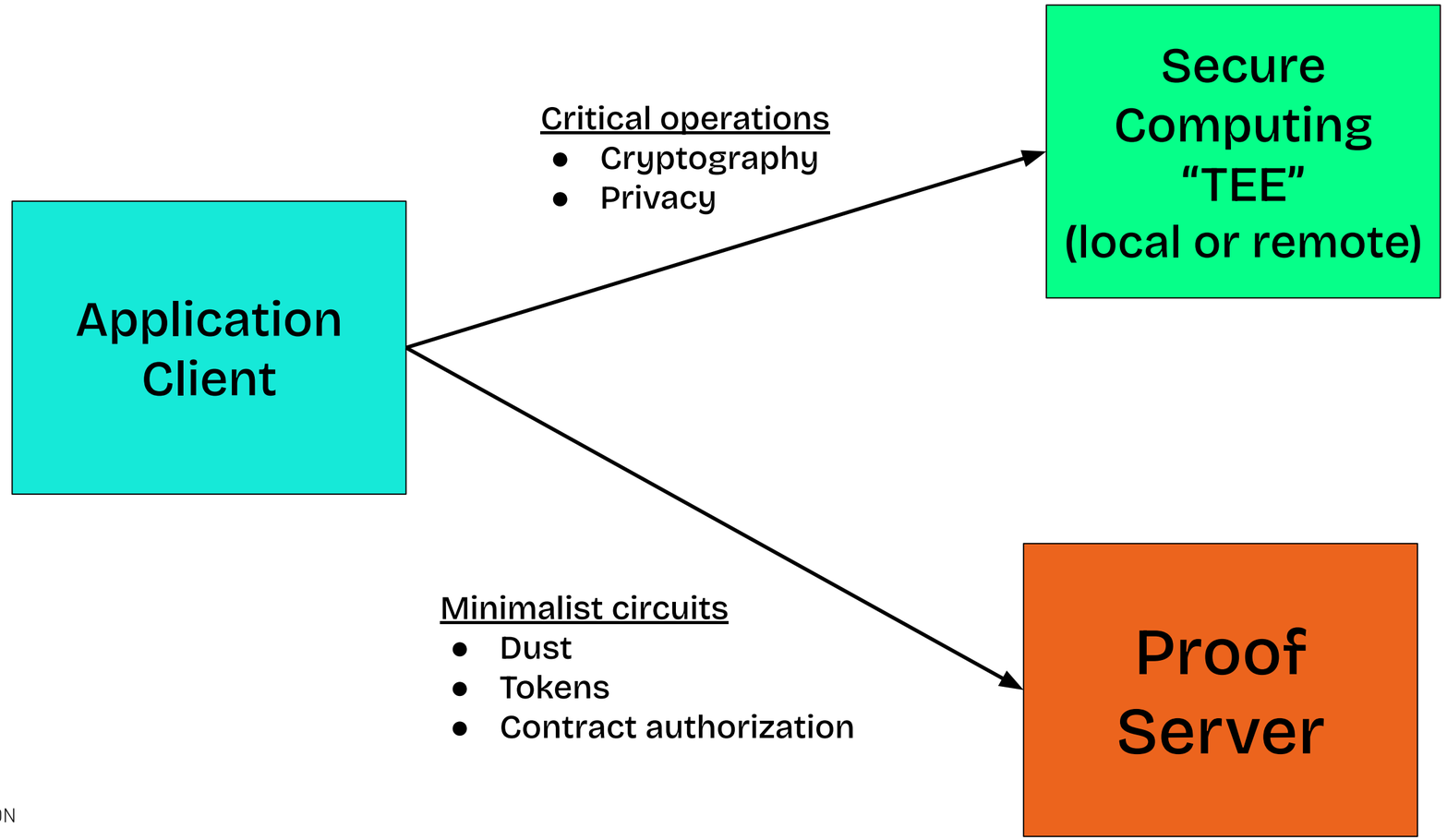
Secure computing: *security vs computation*



Security axis is approximate and multidimensional. Capability categories reflect programmability and addressable memory for cryptographic workloads.

Contracts separating application logic from ZK circuit

Design pattern: “TEE” as an authorization oracle in order to minimize ZK circuit size.

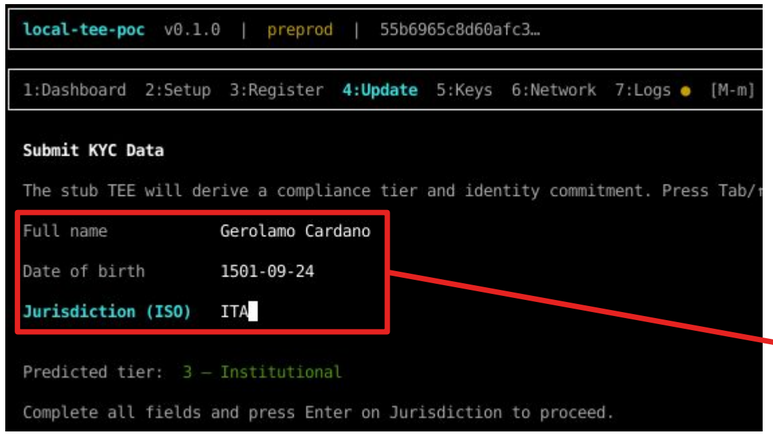


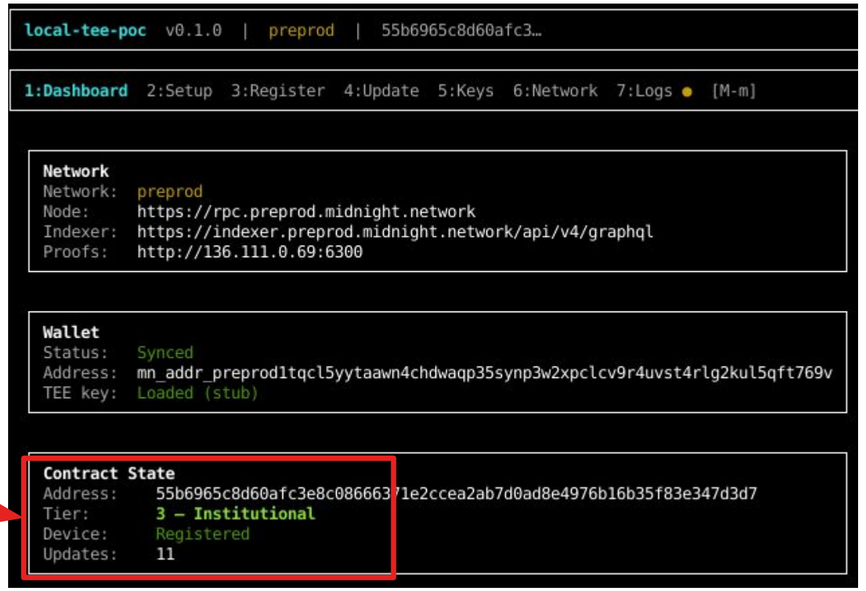
Contracts separating application logic from ZK circuit

Design pattern: “TEE” as an authorization oracle in order to minimize ZK circuit size.

Example: KYC dapp on a mobile device

- After device registration, the Compact contract holds only `device_pk` (public) and `identity_commitment` (private); `sk_device` never leaves the secure enclave (some flavor of TEE).
- `update_compliance` passes only a Schnorr signature (*R*, *s*) to the proof server: the private key stays inside the enclave and merely signs.
- The ZK circuit verifies the signature and enforces the state transition; it contains no application-specific logic.
- *Result:* witness complexity is determined by the authorization scheme (Schnorr ≈ two in-circuit `ecMuls`), not by the authorization logic
- New compliance rules or new business logic → change the TEE code but the ZK circuit remains unchanged and simple.





Proof server timings (outside of hardware TEE)

Circuit	Intel Core Ultra 7 [x86-64 laptop]	Graviton2 (c6g) [arm64 server]	Samsung Tab S7 [Cortex-A77 mobile]	Pixel 9 (est.) [Cortex-X4 mobile]	iPhone 16 Pro (est.) [Apple Everest mobile]
`update_compliance`	1.66 s	15.60 s	6.79 s	~4–5 s	~2.5–3 s
DUST fee	0.62 s	5.53 s	3.62 s	~2.5 s	~1.5 s
Total	*2.28 s*	*21.13 s*	*10.41 s*	*~6–8 s*	*~4–5 s*

- Bottleneck:
  - EC operations like `ecMul` (e.g., Schnorr = 2 × Jubjub scalar multiply) constitutes an in-circuit.
  - Circuits avoiding in-circuit EC ops prove much faster.
- Hard constraint:
  - ZSwap cannot run on-device in consumer-grade secure enclaves due to memory and instruction-set limitations.
  - On higher-end mobile devices, modest ZK proofs can run in the application sandbox, outside of the secure enclave.

SEV-SNP TEE: *Lower-cost alternative to SGX TEE*

Threat eliminated	SEV-SNP	+ Shielded VM	+ Sealed image	+ Port 6300 only	SGX (alternative)
Hypervisor reads VM memory	✅	✅	✅	✅	✅
VM booted from tampered OS	❌	✅	✅	✅	✅
OS tampered interactively	❌	❌	✅	✅	✅
RCE ⟶ post-boot modification	❌	❌	❌	✅	✅
RCE ⟶ read process memory	❌	❌	❌	✅	✅

- SGX is > 3× more costly than SEV-SNP
- SGX audit and configuration is simpler than SEV-SNP
- Neither are available for consumer devices

SEV-SNP experiment

- GCP `n2d-standard-8`
- Ubuntu 22.04
- AMD EPYC Milan
- Proof server operates correctly
- Attestation verified end-to-end via `snpguest`

Findings / hypotheses

1. *From the contract design:*
   - Application logic and ZK circuit complexity can be decoupled.
   - The TEE-as-oracle pattern bounds witness complexity to the authorization scheme alone, a fixed, auditable cost regardless of how the application evolves.
2. *From mobile timing:*
   - For some application-layer circuits, on-device proof generation is feasible in the application sanbox.
   - ZSwap operations and memory footprint are not suited for consumer grade secure enclaves.
3. *From the SEV-SNP experiment:*
   - A sealed-image SEV-SNP VM achieves near-SGX protection at commodity cloud cost.
   - The residual attack surface shrinks to a single binary on a single port — the same trust surface as the code inside an SGX enclave.

Key management

Compact contracts/circuits/libraries and off-chain infrastructure can approximate of NEAR's meta-transaction and multisig patterns with acceptable fidelity and session key and named account UX with caveats around privacy.

Relevant NEAR key management capabilities

Capability	Definition
Named accounts	Human-readable string account IDs (e.g. alice.near) with a hierarchical namespace; a parent account controls creation of sub-accounts (e.g. app.alice.near)
FullAccess keys	Root keypair with authority over all account operations including key rotation, contract deployment, and fund transfer; loss is catastrophic
FunctionCall / session keys	Scoped keypair restricted to a specific contract and method set with an optional NEAR spending allowance; isolates the root key from day-to-day dApp interaction
DelegateAction / meta-tx	User signs an inner transaction payload; a relayer wraps it in an outer transaction and pays the gas; enables gasless onboarding without trusting the relayer with the user's key
HD key derivation	Hierarchical deterministic derivation of an account keypair from a BIP-39 master seed; in NEAR all paths are hardened (Ed25519 limitation), so only the wallet layer benefits — no on-chain derivation
Implicit accounts	Account ID is the hex-encoding of an Ed25519 or secp256k1 public key; the account can receive funds before it is explicitly initialised on-chain, simplifying exchange deposits
Promise-based key mgmt	Contracts can add or delete keys on their *own* account as a promise action; enables social recovery contracts, DAO-controlled custody, and factory-deployed sub-accounts
Chain signatures / MPC	Threshold MPC network (FROST / cait-sith) that holds a distributed key and signs transactions on *other* chains (Ethereum, Bitcoin, etc.) on behalf of a NEAR account; cross-chain execution without bridging assets

NEAR key management usage

Capability	User Value	Corporate Value	App Areas	Usage Frequency	On-Chain Complexity	Off-Chain Complexity
Named accounts	★★★★★	★★★★	All	Universal	Low	Low
FullAccess keys	★★★	★★★	All	Universal	Low	Low
FunctionCall keys	★★★★★	★★★★	dApps, DeFi, enterprise	High	Low	Low
DelegateAction / meta-tx	★★★★	★★★★	Onboarding, all	Growing	Medium	Medium
HD key derivation	★★★★	★★★	All wallets	Universal	None	Low
Ed25519 implicit accounts	★★★	★★	Onboarding, exchanges	Moderate	Low	Low
Promise-based key mgmt	★★	★★★	Recovery, DAO, factory	Moderate	Medium	Medium
Multi-sig contract	★★	★★★★	Treasury, DAO, custody	Moderate	Medium	Medium
Chain Signatures / MPC	★★★★★	★★★★★	Cross-chain DeFi, custody, bridges	Growing	High	High
FastAuth / email recovery	★★★★	★★	Onboarding	Moderate	None	High

Applicability to Midnight

Capability	Midnight has it?	Off-chain cost	Contract cost	Ledger cost	Philosophy fit	Privacy impact	Security impact
Named accounts	No	Low	Moderate	High	Poor	High degradation	Neutral
FullAccess keys	Yes (equivalent)	—	—	—	High	Avoid public key events	Neutral
FunctionCall / session keys	No	Moderate	Moderate-high	Very high	High (redesign for ZK)	Enhancement if ZK-private	High improvement
DelegateAction / meta-tx	No	Moderate	Moderate	High	High	Low risk (public layer)	Low risk
HD key derivation	Wallet-layer only	Very low	N/A	N/A	High	Enhancement (diversified addrs)	High improvement
Implicit accounts	Yes	Low (voucher alt.)	Moderate	High	Poor	High degradation	Introduces footgun
Promise-based key mgmt	No	N/A	Moderate	Very high	High (redesign for ZK)	Enhancement if ZK-private	High improvement
Chain signatures / MPC	No (bilateral Cardano only)	High (MPC infra)	Moderate (interface)	Very high	Very high	Enhancement (private requests)	BN254/BLS12-381 to clarify

Note that the off-chain vs contract vs ledger costs are mutually exclusive: i.e., any one of them would achieve the capability.

Experiment: *Named-account registry with stealth payments*

- **Goal:** Determine whether Midnight can support NEAR-style human-readable names (`alice.midnight`) that also serve as private payment addresses without leaking the recipient's on-chain identity.
- **What was built:** A Compact smart contract registry storing a name plus two public keys (K_{scan}, K_{spend}) on-chain; a full end-to-end ZSwap-shielded transfer path was implemented and exercised on *preprod*.
- **Two-key (scan/spend) design:** Mirrors ERC-5564 and Zcash Sapling.
  - The holder of k_{scan} alone can detect incoming payments but cannot spend, enabling delegation of scanning to a watch-only service without granting spending authority.
  - The holder of both secret keys can spend.
- **Privacy findings**
  - *What is hidden:* Sender identity; token amount; one-time stealth address.
  - *What is leaked:* An observer can see that someone paid something at block *N*.
- Six compact language impediments discovered.

⇒ The stealth meta-address pattern is viable on Midnight, but the Compact language forces workarounds.

Findings / hypotheses

1. Named accounts are feasible via a registry-like contract, coupled with off-chain infrastructure and trusted computing.
2. W3C DIDs may have synergies with named accounts.
3. Contract patterns / circuit libraries can support function-call keys, multisig, and promised-based key management.
4. Metatransactions are consistent with Midnight intents, closely related to work already underway by ARC
5. Midnight already supports full-access keys, HD key derivation, and implicit accounts.

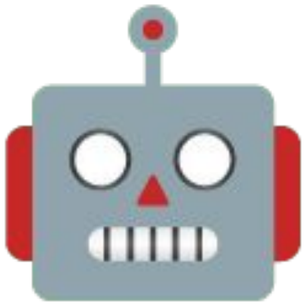
Findings / hypotheses (not covered in these slides)*

NEAR capability	Finding regarding Midnight
Chain abstraction	The integration choice is between reusing NEAR's production MPC and forking to a Midnight-native deployment, but the multi-chain reach is identical in both cases because it derives from the signing curves, not from NEAR's specific infrastructure.
TEE privacy	NEAR's TEE+MPC signing pattern is directly applicable to Midnight but NEAR's threshold ECDSA cannot be reused for Midnight's Schnorr signing requirement, leaving threshold Schnorr to be resolved.
Midnight OS	A Nightstream/Starstream/Paima integration is independent of NEAR and leaves ZSwap unchanged, but it requires a Starstream runtime pallet, and whether Midnight's PLONK/KZG verification layer must also change depends on the proof system chosen for the Nightstream finalizer.
NEAR features	NEAR's compelling user and developer experience/platform can be a model for Midnight.

**These are the subject of other ARC workstreams and some are discussed in detail in the Midnight Passport document.*

Side benefits to other IO and ARC workstreams

LLM-friendly assessments, experiments, and design sketches



Midnight proof server running on mobile (tested with a notional KYC contract)

Bug reports

Lessons learned

```
Midnight TUI  v0.1.0  |  preprod                                   PAUSED  M-m — menu  M-p —

0:Network  1:Dashboard  2:Send  3:Contract  4:Deploy  5:Mint  6:Designate  7:Keys  8:Logs  [M-m]

Chain  preprod
peers 12                ● synced
epoch 986235            next 11m 18s
slot  295870687         13:48:42 UTC
block 188209            hash 0xf5677938f8a9e4d58ff0e6c57582b4d40258db0a17054f743c3bc0f82fad4b4a

Wallet
name         mesonuktion
unshielded   mn_addr_preprod1tqcl5yytaaw
shielded     mn_shield-addr_preprod15xt
dust         mn_dust_preprod1wwwgnz45r

Balances  ● synced
unshielded  NIGHT                                                                  1800.000000
shielded    1702f04984bfca550da6ee3d6b4                                                  7000
shielded    1fdc61cfd6b2640725328c8457f                                                   500
shielded    408e9dbb98d09270da143f4f8906b06c48cf1c19ad7713688d8abeab5957a16d          7000
shielded    5260e3017099ddeeb59062569f0bd6c26a6df9b4d3b69909ceadf1303191be8c          2374
shielded    897a349d064aae5cf23146c55b7fae4f1fd6913208bcdf5e5116994db391b2c6          3000

DUST Generation
balance          5785.078824 DUST
limit            8750.000000 DUST
rate             1249.970400 DUST/day
fill time        fills in 167h 20m
designated       1750.000000 NIGHT
UTXOs            2
```

Full-functioned wallet and contract manager for developers (MN-TUI)

LLMs can and have built specialized applications using this “known good” codebase.

Next steps

1. Retrospective on Tuesday
   a. Claude Code is a participant.
   b. Gemini is the facilitator.
   c. Outbrief shortly afterwards
2. Potential deep dives
   a. TEE, in support of Alan?
   b. TPS experiments and hypothesis testing?
   c. Prototyping key-management contracts/infrastructure?
   d. Assist with roadmapping?
   e. . . . ?